Темплейти (Шаблони).

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Необходими функции в темплейтен клас/ функция. Темплейтни специализации. Примери за темплейтни класове/функции от стандартната библиотека. Умни указатели. Употреба и идея на shared_ptr, weak_ptr, unique_ptr (C++ 11/14) . Пример със класовете Student, School.

Какво са темплейтите?

Темплейтът е механизъм за generic програмиране

Позволява писане на код, независим от типа.

Генерира се код по време на компилация.

```
template<typename T>
T add(T a, T b) {
    return a + b;
}
```

```
add(2, 3);        // T = int
add(2.5, 3.1);    // T = double
```

➡ Типът се извежда по време на компилация

Ограничения

Типът T трябва да поддържа всички използвани операции:

```
template<typename T>
T max(T a, T b) {
    return a > b ? a : b;
}
```

✔ T трябва да има:

operator>

копиране / връщане по стойност

иначе грешка при инстанциране

Какво е темплейтен клас

```
template<typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() const { return value; }
};
Box<int> b1(5);
Box<std::string> b2("Hello");
```

📌 Box<int> и Box<std::string> нямат нищо общо като типове

Пример

```
template<typename T>
class Repository {
    std::vector<T> items;
public:
    void add(const T& t) {
        items.push_back(t);
    }
};
Repository<Student> students;
Repository<int> ids;
```

Какви функции са необходими при шаблонен клас?

Проблем

Зависи как ще се използва шаблонът, но често:
Конструктор
Деструктор
Copy / Move
Оператори (=, ==, <)
const коректност

Грешки при шаблони

Грешките са:
дълги, трудни за четене,
голям и неясен кол стек.

📌 Причина: компилаторът генерира код, който не може да се компилира.

```
template<typename T>
T maxValue(T a, T b) {
    return a > b ? a : b;
}

struct Student {
    int grade;
    // ЛИПСВА operator>
};
```

Грешка

```
int main() {
    Student s1{5};
    Student s2{6};
    auto s = maxValue(s1, s2);
}
```

error: no match for ‘operator>’ (operand types are ‘Student’ and ‘Student’)

Темплейтна специализация

Темплейтната специализация позволява различна имплементация на шаблон за конкретен тип или форма на тип.

„Общ шаблон → специален случай“. Видовете специализация са: Първичен шаблон (Primary template), пълна специализация и частична специализация (само за класове).

```
template<typename T>
struct TypeTraits {
    static void info() {
        std::cout << "Generic type\n";
    }
};//първичен темплейт
```

Пълна специализация – конкретен тип

```
template<>
struct TypeTraits<int> {
    static void info() {
        std::cout << "Integer type\n";
    }
}; //пълна специализация
```

📌 Само за int и напълно заменя първичния template

1)Търси пълна специализация

2)Търси частична специализация (най-специфичната)

3)Използва първичния template

Какво е STL?

STL (Standard Template Library) е част от стандартната библиотека на C++ и представлява колекция от готови, генерични (темплейтни) компоненти за работа с данни и алгоритми.

Основната идея на STL е да раздели данните от алгоритмите и да ги свърже чрез итератори. STL се състои от 4 основни неща:

1)Контейнери – съхраняват данни
2)Алгоритми – обработват данните
3)Итератори – свързват контейнерите с алгоритмите
4)Функционални обекти (functors, lambdas) – поведение/логика

Контейнерите са темплейтни класове, които съхраняват обекти. Биват
Последователни (Sequence containers)

```
std::vector<int> v = {1, 2, 3};
std::list<std::string> names;
```

Какво е STL?

Асоциативни (Associative containers)

```
std::map<int, std::string> idToName;
std::set<int> uniqueValues;
```

Подредени по ключ и Реализирани чрез дървета (log N)

```
std::unordered_map<int, std::string> um; std::unordered_set<int> us;
```

Хеш-базирани, средно O(1) достъп

Алгоритмите са темплейтни функции, които работят с итератори.

```
std::sort(v.begin(), v.end());
std::find(v.begin(), v.end(), 3);
```

Често използвани:

```
std::sort, std::find, std::count, std::copy
```

Итераторите са обобщени указатели.

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}
```

Видове:

```
Input, Output, Forward
```

Какво са smart pointers?

Проблемът с new / delete

Ръчно управление на памет → лесно води до:

1)memory leaks

2)double delete

3)dangling pointers

Особено проблемно при exceptions и сложна логика

Smart pointer = обект, който управлява lifetime-а на друг обект и автоматично освобождава ресурса в деструктора си

Част от STL (<memory>)

Видове

Основни видове (C++11):

std::unique_ptr, std::shared_ptr, std::weak_ptr

std::unique_ptr (единствен собственик)

Само един обект притежава ресурса

Не може да се копира, само да се премества

Най-ефективният smart pointer и избор по подразбиране

Кога се използва

Когато School е единственият собственик на Student

Ясна архитектура и ownership

Пример за unique_ptr

Пример

```
class Student {
public:
    std::string name;
    Student(const std::string& n) : name(n) {}
};
class School {
    std::vector<std::unique_ptr<Student>> students;
public:
    void addStudent(std::unique_ptr<Student> s) {
        students.push_back(std::move(s));
    }
};
```

Видове

```
int main() {

    School school;
school.addStudent(std::make_unique<Student>("
Ivan"));

}
```

Имплементира се като:
Забранено копиране, позволено преместване

Copy constructor и copy assignment са изтрити

=delete

Move constructor и move assignment са позволени

shared_ptr

Един обект може да има няколко собственика

Използва reference counting

Обектът се изтрива, когато броячът стане 0

Кога се използва?

Когато Student се използва от няколко структури

School, Course, External system (например дневник)

Пример

```
class School {
    std::vector<std::shared_ptr<Student>> students;
public:
    void addStudent(const std::shared_ptr<Student>&
s) {
        students.push_back(s);
    }
};
int main() {
    auto st = std::make_shared<Student>("Maria");
    School school;
    school.addStudent(st);
}
```

Проблем със shared_ptr

School държи shared_ptr<Student>

Student държи shared_ptr<School>

```
class Student {
    std::shared_ptr<School> school; // ❌
};
```

Резултат

Reference count никога не става 0

Memory leak

Обектите не се унищожават

Решение

std::weak_ptr

weak_ptr не увеличава reference count

Използва се за:

back-reference,observer, избягване на цикли

```
class School; // forward declaration

class Student {
    std::weak_ptr<School> school; // ✔
public:
    void setSchool(const std::shared_ptr<School>& s) {
        school = s;
    }
};
```

Добри практики

❗ Да се почва почти винаги с unique_ptr

❗ Да се ползва shared_ptr само при нужда

❗ Винаги да се прекъсваат циклите с weak_ptr

✔ Да се ползва make_unique / make_shared (exception safety)

✔ Да се мисли за ownership още при дизайна

std::observer_ptr е официалният стандартен не-притежаващ (non-owning) указателен wrapper. C++ 20

Той:
НЕ притежава паметта
НЕ освобождава (delete)
НЕ управлява жизнения цикъл (lifetime)

Въпроси?

Благодаря за вниманието!